Exercise 1

We use a segment tree with lazy propagation[1] that answers exactly the two types of queries. Building the tree requires $\Theta(n \log n)$ time and $\Theta(n)$ space. Each query takes $\Theta(\log n)$ time and uses $\Theta(\log n)$ space which is freed when the query finishes. Since there are m total queries, the overall time complexity is $\Theta((n+m)\log n)$. The auxiliary space used by the queries does not accumulate, so the overall space complexity is $\Theta(n + \log n) = \Theta(n)$.

Exercise 2

We first compute the frequency array $A[0, n-1]$, where $A[p]$ is the number of segments containing position p.

Next, we construct a collection of lists $\mathrm{pos}[k]$ containing all positions p such that $A[p] = k$. By doing so iterating over the positions p in increasing order and appending each p to $\mathrm{pos}[A[p]]$, we guarantee that every list is sorted in ascending order by construction.

For each query (i, j, k), we perform a binary search on the list $\mathrm{pos}[k]$ to find the first element greater-than-or-equal-to i. If found, we moreover check whether it is less-than-or-equal-to j. If both conditions are satisfied, a valid position has been found; otherwise, none exist.

Computing the arrays A and pos both require $\Theta(n)$ time. Each query (i, j, k) takes $\Theta(\log h)$ time, where h is the length of $\mathrm{pos}[k]$ (which is at most n), yielding $O(\log n)$ per query. Since there are m queries, the overall time complexity is $O(n + m \log n)$. The arrays A and pos both occupy $\Theta(n)$ space: although pos is an array of vectors, it stores a total of n elements across all lists. Overall space complexity is $\Theta(n)$.

[1] The implementation of the segment tree was done following the guide at Algorithms for Competitive Programming .